

Simulazione di Sistemi: Progetto 10

Modello a code a tempo discreto con priorità assoluta
e interruzioni casuali del server

Matteo Pio Trotta

matteopio.trotta@studio.unibo.it

Giugno 2026

Sommario

Il presente documento descrive lo sviluppo, la simulazione e l'analisi prestazionale di un modello a code a tempo discreto caratterizzato da interruzioni casuali del server e da una politica di accodamento basata su classi di priorità assoluta. Il sistema, variante dell'*Option A* studiata da Bruneel e Devos (2025), è stato modellato e simulato in ambiente OMNeT++ 6.3.0 attraverso una campagna di 160 esecuzioni indipendenti su 8 configurazioni parametriche. I risultati empirici sono stati validati contro i modelli analitici della Teoria delle Code, ottenendo un accordo con errori sempre inferiori allo 0.25% sul tempo di permanenza della classe prioritaria e una verifica sistematica della Legge di Little con scarti inferiori allo 0.16% su tutte le configurazioni stabili.

Indice

1	Introduzione e Contesto Scientifico	3
1.1	Il paper di riferimento	3
1.2	Risultati teorici rilevanti per la validazione	3
1.3	Obiettivi del progetto	4
2	Il Modello Analitico	4
2.1	Descrizione del sistema	4
2.2	Caratterizzazione degli arrivi	5
2.3	Dinamica temporale: Late Arrival System	5
2.4	Condizione di stabilità e utilizzo del server	6
3	Implementazione in OMNeT++	6
3.1	Architettura del simulatore	6
3.2	Logica del modulo C++	7
3.3	Configurazione della campagna	8
4	Analisi del Transiente Iniziale	9
4.1	Il problema	9
4.2	Stima empirica con il metodo di Welch	9
4.3	Integrazione con il motore statistico	9

5	Metodologia Statistica	9
5.1	Repliche indipendenti	9
5.2	Intervalli di confidenza con t di Student	10
6	Risultati e Analisi Approfondita	10
6.1	Throughput e verifica della stabilità	10
6.2	Validazione analitica del tempo di permanenza della coda 1	11
6.3	Tempo medio di permanenza: stime complete	11
6.4	Lunghezza media delle code e verifica della Legge di Little	12
7	Conclusioni	13
A	Codice Sorgente Completo (C++)	14
B	Topologia NED	15
C	Configurazione omnetpp.ini	16
D	Script di Analisi Statistica (Python)	16

1 Introduzione e Contesto Scientifico

1.1 Il paper di riferimento

Il fondamento teorico del progetto è l'articolo:

H. Bruneel, A. Devos, “*Coupled queues with server interruptions: Some solutions*”, Performance Evaluation, vol. 167, 2025, art. 102466.
<https://doi.org/10.1016/j.peva.2024.102466>

Gli autori studiano una famiglia di sistemi a code a tempo discreto che combinano tre caratteristiche raramente analizzate congiuntamente nella letteratura: *code accoppiate* (coupled queues), *interruzioni casuali del server* e *scheduling a priorità*. Tutti i modelli condividono le seguenti ipotesi: due classi di utenti (tipo 1 e tipo 2) accodate in due code separate a capacità infinita, tempi di servizio deterministici pari a un time slot, e arrivi indipendenti da slot a slot ma potenzialmente correlati tra le classi all'interno dello stesso slot, descritti dalla funzione generatrice di probabilità (pgf) congiunta $A(z_1, z_2)$.

L'articolo analizza tre diverse architetture della facility di servizio:

Option A: un *unico server condiviso* dalle due code, soggetto a interruzioni casuali di tipo Bernoulli (disponibile con probabilità σ in ogni slot), che applica **priorità assoluta** agli utenti di tipo 1. Per questo modello gli autori derivano una equazione funzionale per la pgf congiunta stazionaria $U(z_1, z_2)$ del contenuto delle due code (Eq. 14 del paper) e la risolvono in forma chiusa per $A(z_1, z_2)$ arbitraria mediante il teorema di Rouché.

Option B: due server dedicati (uno per coda) con interruzioni *simultanee*.

Option C: due server dedicati, con interruzioni solo sul server della coda 2.

Il sistema oggetto del presente progetto è una **variante dell'Option A**: la differenza rispetto al modello generale del paper è che gli arrivi delle due classi sono assunti *statisticamente indipendenti* tra loro, ossia $A(z_1, z_2) = A_1(z_1) \cdot A_2(z_2)$, con marginali geometrica e di Poisson rispettivamente (si veda la Sezione 2.2).

1.2 Risultati teorici rilevanti per la validazione

Dal paper di riferimento (Sezioni 3.1 e 4) ricaviamo i risultati analitici utilizzati per validare il simulatore:

Stabilità (Eq. 4 del paper). L'Option A è *work-conserving*: il server, quando disponibile, non resta mai inattivo se c'è almeno un utente nel sistema. Ne consegue che il sistema è stabile se e solo se

$$\lambda_T = \lambda_1 + \lambda_2 < \sigma. \quad (1)$$

Isolamento della classe prioritaria (Eq. 5 del paper). Grazie alla priorità assoluta, il contenuto della coda 1 non è in alcun modo influenzato dalla presenza di utenti di tipo 2. La pgf marginale stazionaria $U_1(z_1)$ coincide con quella di una coda singola a tempo discreto con arrivi $A_1(z_1)$, servizio unitario e interruzioni Bernoulli:

$$U_1(z_1) = \frac{(\sigma - \lambda_1)(z_1 - 1)A_1(z_1)}{z_1 - [\sigma + (1 - \sigma)z_1]A_1(z_1)}. \quad (2)$$

Questo risultato è la chiave della nostra validazione: derivando la (2) e applicando la Legge di Little si ottiene una formula chiusa per il tempo medio di permanenza della classe 1 (Sezione 6.2).

Condizione necessaria per la coda 2. Quando $\lambda_T > \sigma$ ma $\lambda_1 < \sigma$, la coda 1 resta stabile mentre la coda 2 diverge. In tale regime il throughput della classe 2 si satura alla capacità residua del server:

$$\theta_2^{\text{sat}} = \sigma - \lambda_1. \quad (3)$$

1.3 Obiettivi del progetto

Gli obiettivi, definiti dalla traccia, sono:

1. **Modellazione:** tradurre il modello analitico (variante dell'Option A) in un simulatore a eventi discreti sulla piattaforma OMNeT++;
2. **Campagna sperimentale:** esplorare tutte le combinazioni dei parametri $p \in \{0.75, 0.85\}$, $q \in \{0.2, 0.3\}$, $\sigma \in \{0.6, 0.7\}$, per un totale di 8 configurazioni, con *almeno 20 run indipendenti* ciascuna;
3. **Stima delle prestazioni:** per ciascuna classe $i \in \{1, 2\}$, fornire stima puntuale e intervallo di confidenza per (a) il tempo medio di permanenza $E[W_i]$, (b) il throughput θ_i e (c) la lunghezza media della coda $E[L_i]$;
4. **Analisi del transiente iniziale:** individuare e rimuovere metodologicamente il bias di inizializzazione (warm-up);
5. **Validazione:** confrontare i risultati simulativi con le previsioni del modello analitico di riferimento.

2 Il Modello Analitico

2.1 Descrizione del sistema

Il sistema è costituito da un singolo server e due code a capacità infinita. L'asse temporale è suddiviso in slot di durata fissa unitaria. Le regole di funzionamento sono:

- Gli utenti di tipo i ($i = 1, 2$) vengono accodati nella coda i ;
- Il servizio di ogni utente, indipendentemente dalla classe, dura esattamente uno slot;
- **Priorità assoluta:** se il server è disponibile, serve un utente della coda 1 se questa è non vuota; serve la coda 2 *solo* se la coda 1 è completamente vuota;

- **Interruzioni Bernoulli:** in ogni slot il server è disponibile con probabilità σ , indipendentemente da slot a slot e dallo stato delle code. Le interruzioni iniziano e terminano ai bordi degli slot, quindi non interrompono mai un servizio in corso (i servizi durano esattamente uno slot).

Si noti che, come osservato nel paper (Sezione 2.3), con servizi deterministici di un solo slot la distinzione tra priorità *preemptive* e *non-preemptive* è irrilevante: un servizio non può mai essere interrotto da un arrivo a priorità superiore, essendo i servizi sincronizzati ai bordi degli slot.

2.2 Caratterizzazione degli arrivi

Sia u_i la variabile casuale che descrive il numero di arrivi di classe i in un generico slot. I due processi di arrivo sono indipendenti tra loro e da slot a slot.

Classe 1 – Geometrica. Il numero di arrivi di tipo 1 segue una distribuzione geometrica di parametro p , con supporto $\{0, 1, 2, \dots\}$:

$$A_1(u_1) = p(1-p)^{u_1}, \quad u_1 = 0, 1, 2, \dots \quad (4)$$

Il valore atteso è

$$\lambda_1 = E[u_1] = \frac{1-p}{p}. \quad (5)$$

Per $p = 0.75$ si ha $\lambda_1 = 1/3 \approx 0.3333$; per $p = 0.85$, $\lambda_1 \approx 0.1765$. Si noti che valori *più alti* di p corrispondono a un traffico *minore*.

Classe 2 – Poisson. Il numero di arrivi di tipo 2 segue una distribuzione di Poisson di parametro q :

$$A_2(u_2) = \frac{q^{u_2}}{u_2!} e^{-q}, \quad u_2 = 0, 1, 2, \dots \quad (6)$$

con valore atteso $\lambda_2 = E[u_2] = q$.

2.3 Dinamica temporale: Late Arrival System

Nei sistemi a tempo discreto è essenziale definire rigorosamente l'ordine degli eventi all'interno dello slot. Abbiamo adottato la semantica *Late Arrival System* (LAS), coerente con le system equations (11)–(12) del paper di riferimento:

1. **Fase di servizio** (inizio slot k): viene campionata la disponibilità del server; se disponibile, viene servito un utente *già presente* in coda all'istante k^- , secondo la regola di priorità;
2. **Fase di arrivo** (interno dello slot k): vengono generati i nuovi arrivi, che vengono accodati ma diventano eleggibili per il servizio non prima dell'inizio dello slot $k+1$.

Questa scelta vincola il tempo di permanenza minimo di ogni utente a un intero slot e garantisce la coerenza con le system equations del modello analitico, nelle quali $u_{i,k+1} = a_{i,k} + (\text{contenuto residuo dopo il servizio})$.

2.4 Condizione di stabilità e utilizzo del server

Essendo il sistema work-conserving, vale la condizione (1). È utile definire il fattore di utilizzo

$$\rho = \frac{\lambda_T}{\sigma}, \quad (7)$$

che misura la frazione di capacità effettiva del server impegnata dal traffico totale. La Tabella 1 riassume ρ per le 8 configurazioni della campagna: si nota che una sola configurazione ($p = 0.75$, $q = 0.3$, $\sigma = 0.6$) presenta $\rho > 1$ ed è quindi instabile per la coda 2.

Tabella 1: Carico offerto e fattore di utilizzo per le 8 configurazioni

p	q	σ	λ_1	λ_T	$\rho = \lambda_T/\sigma$
0.85	0.2	0.7	0.1765	0.3765	0.538
0.85	0.2	0.6	0.1765	0.3765	0.627
0.85	0.3	0.7	0.1765	0.4765	0.681
0.75	0.2	0.7	0.3333	0.5333	0.762
0.85	0.3	0.6	0.1765	0.4765	0.794
0.75	0.2	0.6	0.3333	0.5333	0.889
0.75	0.3	0.7	0.3333	0.6333	0.905
0.75	0.3	0.6	0.3333	0.6333	1.056

La campagna copre dunque uno spettro di carichi molto ampio, da $\rho \approx 0.54$ (sistema scarico) fino a $\rho \approx 1.06$ (sovraccarico), permettendo di osservare il comportamento del sistema in regimi qualitativamente diversi, incluso il collasso della coda a bassa priorità.

3 Implementazione in OMNeT++

3.1 Architettura del simulatore

Il simulatore è stato sviluppato su OMNeT++ 6.3.0 ed è composto da tre file:

`SistemaPriorita.ned` definisce la topologia: un'unica rete (`SistemaPriorita`) contenente un singolo modulo semplice (`ModuloCode`) che incapsula l'intera logica del nodo servente: le due code, il server con interruzioni e i generatori di arrivi. Nello stesso file sono dichiarati i 6 segnali (`@signal`) e le 6 statistiche (`@statistic`) per la raccolta delle metriche, in modo indipendente per le due code;

`ModuloCode.cc` implementa in C++ il comportamento slot-per-slot (Sezione 3.2);

`omnetpp.ini` orchestra la campagna di simulazione in modalità batch (Sezione 3.3).

La scelta di un singolo modulo, anziché moduli separati per sorgenti, code e server, è motivata dalla natura strettamente sincrona del modello: tutti gli eventi avvengono ai bordi degli slot e l'ordine servizio→arrivi all'interno dello slot è più semplice e robusto da garantire all'interno di un'unica `handleMessage`, evitando problemi di ordinamento di eventi simultanei tra moduli diversi.

3.2 Logica del modulo C++

Scansione del tempo discreto. Un auto-messaggio (`slotEvent`) viene rischedulato esattamente ogni 1.0 s di tempo simulato: ogni sua ricezione segna l'inizio di un nuovo time slot.

Strutture dati. Per stimare il tempo di permanenza di ciascun utente non basta contare gli utenti presenti: occorre preservarne l'identità temporale. Le due code sono quindi modellate come `std::queue<simtime_t>`, nelle quali ogni elemento è il *timestamp di arrivo* dell'utente. Al momento del servizio, il delay si ottiene come differenza `simTime() - arrivalTime`. La disciplina FIFO della `std::queue` riflette l'ordine di servizio all'interno della classe.

Ordine delle operazioni nello slot. La funzione `handleMessage` esegue, nell'ordine: (1) la fase di servizio sugli utenti preesistenti, (2) la generazione dei nuovi arrivi, (3) l'emissione delle statistiche, (4) la rischedulazione dello slot successivo. Questo ordine implementa esattamente la semantica LAS della Sezione 2.3: un utente generato al passo (2) non può essere servito nello stesso slot, perché la fase di servizio (1) è già avvenuta.

```
1 virtual void handleMessage(cMessage *msg) override {
2     int served1 = 0, served2 = 0;
3
4     // --- 1. SERVIZIO (solo utenti preesistenti) ---
5     bool serverAvailable = uniform(0, 1) < sigma;
6     if (serverAvailable) {
7         if (!queue1.empty()) { // Priorita' assoluta
8             simtime_t arrivalTime = queue1.front();
9             queue1.pop();
10            emit(q1DelaySignal, simTime() - arrivalTime);
11            served1 = 1;
12        } else if (!queue2.empty()) { // Coda 2 solo se Coda 1
13            vuota
14            simtime_t arrivalTime = queue2.front();
15            queue2.pop();
16            emit(q2DelaySignal, simTime() - arrivalTime);
17            served2 = 1;
18        }
19    }
20
21    // --- 2. NUOVI ARRIVI ---
22    int a1 = geometric(p); // Geometrica, supporto
23    {0,1,2,...}
24    for (int i = 0; i < a1; i++) queue1.push(simTime());
25
26    int a2 = poisson(q); // Poisson
27    for (int i = 0; i < a2; i++) queue2.push(simTime());
28
29    // --- 3. STATISTICHE ---
30    emit(q1LenSignal, (int)queue1.size());
31    emit(q2LenSignal, (int)queue2.size());
32    emit(q1ThroughputSignal, served1);
33    emit(q2ThroughputSignal, served2);
34
35    scheduleAt(simTime() + 1.0, slotEvent);
36 }
```

Listing 1: Cuore della logica di simulazione (`handleMessage`)

Generazione degli arrivi. La funzione di libreria `geometric(p)` di OMNeT++ genera valori interi con supporto $\{0, 1, 2, \dots\}$ e media $(1-p)/p$, ossia esattamente la distribuzione (4) richiesta dalla traccia. Analogamente `poisson(q)` implementa la (6). Entrambe le funzioni attingono ai generatori Mersenne Twister di OMNeT++, con seed automaticamente differenziati tra le repliche (`repetition`), garantendo l'indipendenza statistica dei 20 run.

Raccolta delle metriche. Le tre misure di prestazione sono catturate con strategie diverse, scelte per delegare al motore statistico di OMNeT++ l'aggregazione corretta (inclusa la rimozione del transiente):

- **Lunghezza delle code:** a fine slot viene emessa la dimensione istantanea `size()` delle due `std::queue`; la media temporale di questo segnale campionato a passo costante coincide con $E[L_i]$;
- **Tempo di permanenza:** emesso al completamento del servizio come differenza tra tempo corrente e timestamp di arrivo; la media aritmetica dei campioni è la stima di $E[W_i]$;
- **Throughput:** registrato come variabile binaria per slot (1 se un utente della classe è stato servito, 0 altrimenti). Il valore atteso di questa sequenza di Bernoulli coincide con il numero medio di utenti serviti per slot, cioè con θ_i .

3.3 Configurazione della campagna

Il file `omnetpp.ini` sfrutta la sintassi di iterazione di OMNeT++ per generare automaticamente tutte le combinazioni dei parametri:

```
1 [General]
2 network = SistemaPriorita
3 sim-time-limit = 200000s
4 repeat = 20
5 warmup-period = 10000s
6
7 **.p      = ${p_val=0.75, 0.85}
8 **.q      = ${q_val=0.2, 0.3}
9 **.sigma = ${sigma_val=0.6, 0.7}
```

Il prodotto cartesiano dei tre insiemi genera $2 \times 2 \times 2 = 8$ configurazioni; con `repeat = 20` si ottengono $8 \times 20 = 160$ run totali, ciascuno di 200 000 slot. L'esecuzione è avvenuta in modalità batch (`Cmdenv`):

```
1 ./Progetto10.exe -m -u Cmdenv -c General omnetpp.ini
```

con esito `Run statistics: total 160, successful 160`.

4 Analisi del Transiente Iniziale

4.1 Il problema

All'istante $t = 0$ il sistema parte vuoto: entrambe le code contengono zero utenti. Questa condizione iniziale non è rappresentativa del regime stazionario: durante i primi slot le code devono “riempirsi” fino ad assorbire la dinamica di arrivi e interruzioni. Se i campioni del transiente venissero inclusi nelle medie, le stime risulterebbero sistematicamente *distorte verso il basso* (initialization bias), dato che il sistema scarico esibisce delay e lunghezze minori di quelle a regime.

4.2 Stima empirica con il metodo di Welch

La durata del warm-up non è stata scelta arbitrariamente, ma valutata empiricamente con il metodo delle *Welch's Moving Averages*: si eseguono R repliche indipendenti, si calcola la media tra repliche del processo osservato a ogni slot, e si applica una media mobile per filtrare il rumore residuo; il warm-up termina dove la curva si assesta attorno a un valore costante.

L'analisi è stata condotta sulla metrica più lenta a convergere: la **lunghezza della coda 2** nella configurazione stabile a maggior carico ($p = 0.75$, $q = 0.2$, $\sigma = 0.6$, $\rho \approx 0.89$). La coda 2 è la più critica perché, essendo a bassa priorità, vede una capacità residua piccola e ha quindi il tempo di rilassamento più lungo dell'intero sistema. A tale scopo è stata definita nel file `ini` una configurazione dedicata (`[Config TransienteWelch]`) che registra la storia temporale completa (vettore) del segnale `q2LenSignal` su 20 repliche.

L'osservazione delle medie mobili ha mostrato che il processo esaurisce il transitorio e si assesta ben prima dei 10 000 slot. Per garantire un margine cautelativo, il warm-up è stato fissato a:

`warmup-period = 10000s`

4.3 Integrazione con il motore statistico

La direttiva `warmup-period` agisce in sinergia con l'architettura a segnali: il motore di OMNeT++ scarta automaticamente *tutti* i valori emessi nei primi 10 000 slot — inclusi i campioni binari del throughput — senza alcuna logica di reset hard-coded nel C++. Su una durata totale di 200 000 slot, le statistiche vengono quindi calcolate sui 190 000 slot finali, pienamente rappresentativi del regime stazionario. Il rapporto warm-up/durata totale (5%) garantisce che il costo della rimozione del transiente sia trascurabile rispetto alla precisione guadagnata.

5 Metodologia Statistica

5.1 Repliche indipendenti

Per ciascuna delle 8 configurazioni sono state eseguite $N = 20$ repliche indipendenti (metodo delle *independent replications*). Ogni replica usa seed differenti per i generatori pseudo-casuali e produce una stima puntuale per ciascuna metrica (la media intra-run calcolata dal motore di OMNeT++ sul periodo post-warm-up). Le 20 stime $X_1, \dots, X_{20}$

così ottenute sono variabili i.i.d., requisito necessario per la costruzione di intervalli di confidenza validi.

5.2 Intervalli di confidenza con t di Student

Essendo il campione di dimensione ridotta ($N = 20 < 30$) e la varianza della popolazione ignota, l'uso della Normale standard non è giustificato: si impiega la distribuzione t di Student con $\nu = N - 1 = 19$ gradi di libertà. Detta $\bar{X}$ la media campionaria e S la deviazione standard campionaria, l'intervallo di confidenza al 95% è:

$$\text{IC}_{95\%} = \bar{X} \pm t_{0.025, 19} \cdot \frac{S}{\sqrt{N}}, \quad t_{0.025, 19} \approx 2.093. \quad (8)$$

L'estrazione dei dati dai 160 file `.sca` e il calcolo di medie e IC sono automatizzati dallo script `analisi_risultati.py` (incluso nel repository del progetto), che esegue il parsing delle righe `scalar` e applica la (8) a tutte le metriche di tutte le configurazioni.

6 Risultati e Analisi Approfondita

6.1 Throughput e verifica della stabilità

Tabella 2: Throughput simulato a regime e confronto con il carico offerto

p	q	σ	λ_1	λ_2	λ_T	θ_1	θ_2	Stato
0.75	0.2	0.6	0.3333	0.2	0.5333	0.3332	0.1998	Stabile
0.75	0.2	0.7	0.3333	0.2	0.5333	0.3336	0.2000	Stabile
0.75	0.3	0.6	0.3333	0.3	0.6333	0.3332	0.2670	Instabile (C2)
0.75	0.3	0.7	0.3333	0.3	0.6333	0.3337	0.2997	Stabile
0.85	0.2	0.6	0.1765	0.2	0.3765	0.1766	0.2003	Stabile
0.85	0.2	0.7	0.1765	0.2	0.3765	0.1767	0.1998	Stabile
0.85	0.3	0.6	0.1765	0.3	0.4765	0.1764	0.3001	Stabile
0.85	0.3	0.7	0.1765	0.3	0.4765	0.1767	0.3005	Stabile

Nelle 7 configurazioni stabili il throughput misurato coincide con il tasso di arrivo ($\theta_i \approx \lambda_i$) con scarti inferiori allo 0.3%: tutto il traffico offerto viene smaltito, come deve avvenire in un sistema sotto la soglia di saturazione.

Analisi del caso instabile. La configurazione ($p = 0.75$, $q = 0.3$, $\sigma = 0.6$) ha $\lambda_T = 0.6333 > 0.6 = \sigma$, dunque viola la (1). Il comportamento osservato conferma puntualmente la teoria:

- la **coda 1 resta stabile**: poiché $\lambda_1 = 0.3333 < \sigma$, la classe prioritaria continua a essere servita regolarmente e il suo throughput resta $\theta_1 = 0.3332 \approx \lambda_1$, del tutto indifferente al collasso della coda 2;
- la **coda 2 si satura** alla capacità residua: il valore teorico (3) è $\theta_2^{\text{sat}} = 0.6 - 0.3333 = 0.2667$, mentre la simulazione misura 0.2670, con un errore dello 0.13%. Il deficit $\lambda_2 - \theta_2 = 0.3 - 0.267 = 0.033$ utenti/slot si accumula indefinitamente nella coda, che cresce linearmente nel tempo.

6.2 Validazione analitica del tempo di permanenza della coda 1

Dalla pgf (2), derivando in $z_1 = 1$ e applicando la Legge di Little, con arrivi geometrici si ottiene la formula chiusa:

$$E[W_1] = \frac{1}{\sigma - \lambda_1}. \quad (9)$$

Tabella 3: Validazione: $E[W_1]$ analitico vs simulato (tutte le configurazioni stabili per la coda 1)

p	q	σ	$E[W_1]$ teorico	$E[W_1]$ simulato	Errore (%)
0.75	0.2	0.6	3.7500	3.7469	0.08
0.75	0.2	0.7	2.7273	2.7322	0.18
0.75	0.3	0.6	3.7500	3.7497	0.01
0.75	0.3	0.7	2.7273	2.7319	0.17
0.85	0.2	0.6	2.3611	2.3604	0.03
0.85	0.2	0.7	1.9101	1.9126	0.13
0.85	0.3	0.6	2.3611	2.3622	0.05
0.85	0.3	0.7	1.9101	1.9146	0.23

L'accordo è eccellente su *tutte e otto* le configurazioni (errore massimo 0.23%), inclusa quella instabile per la coda 2 — ulteriore conferma sperimentale dell'isolamento della classe prioritaria: la (9) resta valida anche quando la coda 2 diverge, perché dipende solo da λ_1 e σ .

Si osservi inoltre, confrontando le righe a parità di (p, σ) , che $E[W_1]$ è **del tutto insensibile a q** : ad esempio con $p = 0.75$, $\sigma = 0.7$ si misura 2.7322 per $q = 0.2$ e 2.7319 per $q = 0.3$ — differenza ben dentro gli intervalli di confidenza. Questo è il “perfetto isolamento” della classe prioritaria previsto dal modello analitico.

6.3 Tempo medio di permanenza: stime complete

Tabella 4: Tempo medio di permanenza $E[W_i]$ [slot], stima puntuale e IC al 95%

p	q	σ	$E[W_1] \pm \text{IC}$	$E[W_2] \pm \text{IC}$
0.75	0.2	0.6	3.7469 ± 0.0184	27.186 ± 0.703
0.75	0.2	0.7	2.7322 ± 0.0130	8.897 ± 0.096
0.75	0.3	0.6	3.7497 ± 0.0276	∞ (instabile)*
0.75	0.3	0.7	2.7319 ± 0.0103	21.315 ± 0.379
0.85	0.2	0.6	2.3604 ± 0.0087	5.100 ± 0.045
0.85	0.2	0.7	1.9126 ± 0.0046	3.292 ± 0.019
0.85	0.3	0.6	2.3622 ± 0.0110	8.782 ± 0.109
0.85	0.3	0.7	1.9146 ± 0.0069	4.554 ± 0.031

*Il simulatore misura $\approx 11\,600$ slot, valore privo di significato stazionario: cresce con la durata della simulazione.

Effetto del traffico di bassa priorità (q) sul delay della coda 2. A parità di p e σ , l'incremento di q da 0.2 a 0.3 produce un aumento di $E[W_2]$ fortemente *non lineare*, tanto più violento quanto più il sistema è vicino alla saturazione:

- $p = 0.85, \sigma = 0.7$ ($\rho : 0.54 \rightarrow 0.68$): $E[W_2]$ passa da 3.29 a 4.55 (+38%);
- $p = 0.85, \sigma = 0.6$ ($\rho : 0.63 \rightarrow 0.79$): da 5.10 a 8.78 (+72%);
- $p = 0.75, \sigma = 0.7$ ($\rho : 0.76 \rightarrow 0.90$): da 8.90 a 21.32 (+140%);
- $p = 0.75, \sigma = 0.6$ ($\rho : 0.89 \rightarrow 1.06$): da 27.19 alla *divergenza*.

Questa progressione è la firma classica della crescita iperbolica del ritardo in funzione del carico ($E[W] \propto 1/(\sigma - \lambda_T)$ in prossimità della saturazione): un incremento del 50% del traffico di classe 2 può più che raddoppiare il suo ritardo, fino a determinarne il collasso. Nel frattempo, in tutti e quattro i confronti, $E[W_1]$ resta identico entro l'errore statistico.

Effetto della disponibilità del server (σ). A parità di carico, aumentare σ da 0.6 a 0.7 (cioè +16.7% di capacità) produce benefici asimmetrici tra le classi:

- per la **coda 1** la riduzione del delay è quella prevista dalla (9): -27.1% con $p = 0.75$ e -19.0% con $p = 0.85$;
- per la **coda 2** la riduzione è molto più marcata: -35.5% ($p = 0.85, q = 0.2$), -48.1% ($p = 0.85, q = 0.3$), -67.3% ($p = 0.75, q = 0.2$), fino al recupero della stabilità nel caso $p = 0.75, q = 0.3$.

L'asimmetria si spiega osservando che la coda 2 beneficia *due volte* dell'aumento di capacità: direttamente (più slot disponibili) e indirettamente (la coda 1 si svuota più in fretta, lasciando più slot residui alla classe non prioritaria). La classe a bassa priorità è quindi la più sensibile, in entrambe le direzioni, alle variazioni di capacità del server.

6.4 Lunghezza media delle code e verifica della Legge di Little

Tabella 5: Lunghezza media delle code $E[L_i]$, stima puntuale e IC al 95%

p	q	σ	$E[L_1] \pm \text{IC}$	$E[L_2] \pm \text{IC}$
0.75	0.2	0.6	1.2486 ± 0.0077	5.4336 ± 0.1431
0.75	0.2	0.7	0.9116 ± 0.0056	1.7789 ± 0.0193
0.75	0.3	0.6	1.2493 ± 0.0119	∞ (instabile)
0.75	0.3	0.7	0.9116 ± 0.0042	6.3889 ± 0.1152
0.85	0.2	0.6	0.4169 ± 0.0020	1.0216 ± 0.0102
0.85	0.2	0.7	0.3379 ± 0.0013	0.6578 ± 0.0039
0.85	0.3	0.6	0.4166 ± 0.0025	2.6357 ± 0.0347
0.85	0.3	0.7	0.3383 ± 0.0018	1.3684 ± 0.0104

Verifica sistematica della Legge di Little. La relazione $E[L_i] = \lambda_i \cdot E[W_i]$ lega due metriche stimate da segnali completamente diversi (la lunghezza è campionata a fine slot, il delay è misurato per-utente al completamento del servizio): la sua verifica costituisce quindi un test di consistenza interna molto forte del simulatore. La Tabella 6 mostra il confronto su tutte le configurazioni stabili.

Tabella 6: Verifica della Legge di Little su entrambe le code (configurazioni stabili)

p	q	σ	$E[L_1]$	$\lambda_1 E[W_1]$	err. %	$E[L_2]$	$\lambda_2 E[W_2]$	err. %
0.75	0.2	0.6	1.2486	1.2490	0.03	5.4336	5.4373	0.07
0.75	0.2	0.7	0.9116	0.9107	0.10	1.7789	1.7793	0.02
0.75	0.3	0.7	0.9116	0.9106	0.11	6.3889	6.3946	0.09
0.85	0.2	0.6	0.4169	0.4165	0.09	1.0216	1.0200	0.16
0.85	0.2	0.7	0.3379	0.3375	0.11	0.6578	0.6583	0.08
0.85	0.3	0.6	0.4166	0.4169	0.06	2.6357	2.6346	0.04
0.85	0.3	0.7	0.3383	0.3379	0.13	1.3684	1.3662	0.16

Gli scarti, mai superiori allo 0.16%, sono pienamente compatibili con la sola variabilità statistica residua: il simulatore è internamente consistente.

Precisione delle stime. Gli intervalli di confidenza della coda 1 sono nell’ordine dei millesimi (IC relativo $< 1\%$ ovunque), grazie all’elevato numero di campioni utili per run ($\sim 190\,000$ slot) e alla bassa varianza della classe prioritaria. Gli IC della coda 2 sono più ampi in termini relativi nelle configurazioni cariche (fino a $\sim 2.6\%$ per $p = 0.75$, $q = 0.2$, $\sigma = 0.6$): è l’atteso effetto dell’aumento della varianza e dell’autocorrelazione del processo di coda in prossimità della saturazione, che riduce il numero effettivo di campioni indipendenti per run.

Ruolo di buffer della coda 2. In tutti gli scenari la coda 1 resta corta (da 0.34 a 1.25 utenti medi): la priorità assoluta la mantiene prossima allo svuotamento. L’intera variabilità del sistema si scarica sulla coda 2, che funge da “ammortizzatore”: a parità di carico totale, il suo contenuto cresce in modo iperbolico con ρ fino a perdere significato stazionario nella configurazione satura, dove la simulazione misura lunghezze medie nell’ordine delle migliaia di utenti, crescenti con l’orizzonte di osservazione.

7 Conclusioni

Il progetto ha portato allo sviluppo di un simulatore a eventi discreti su OMNeT++ per un sistema a code a tempo discreto con priorità assoluta e interruzioni Bernoulli del server, variante a arrivi indipendenti dell’Option A di Bruneel e Devos (2025). La campagna di 160 run su 8 configurazioni, con warm-up di 10 000 slot dimensionato empiricamente con il metodo di Welch e intervalli di confidenza al 95% basati sulla t di Student, ha prodotto i seguenti risultati:

1. **Correttezza del simulatore:** il tempo medio di permanenza della classe prioritaria coincide con la formula analitica $E[W_1] = 1/(\sigma - \lambda_1)$ con errore massimo dello 0.23% su tutte le 8 configurazioni;
2. **Consistenza interna:** la Legge di Little è verificata su entrambe le code in tutte le configurazioni stabili, con scarti $\leq 0.16\%$;
3. **Isolamento della classe prioritaria:** delay e lunghezza della coda 1 sono sperimentalmente indipendenti dal traffico di classe 2, anche quando la coda 2 diverge — conferma del risultato strutturale del modello analitico;

4. **Comportamento al collasso:** nella configurazione satura ($\lambda_T = 0.633 > \sigma = 0.6$) il throughput della coda 2 si assesta esattamente alla capacità residua $\sigma - \lambda_1 = 0.2667$ (errore 0.13%), mentre le sue metriche stazionarie perdono significato;
5. **Sensibilità ai parametri:** la classe a bassa priorità è molto più sensibile della classe prioritaria sia all'aumento del proprio traffico (crescita iperbolica del delay, fino a +140% a fronte di +50% di q) sia alle variazioni di capacità del server (riduzioni del delay fino al 67% per +16.7% di σ).

Possibili estensioni future includono la simulazione delle Option B e C del paper (server dedicati con interruzioni simultanee o asimmetriche), l'introduzione di arrivi correlati tra le classi tramite pgf congiunta non fattorizzabile, e modelli di interruzione più generali (on/off markoviano) rispetto al semplice schema Bernoulli.

A Codice Sorgente Completo (C++)

```

1  #include <omnetpp.h>
2  #include <queue>
3
4  using namespace omnetpp;
5
6  class ModuloCode : public cSimpleModule {
7  private:
8      std::queue<simtime_t> queue1;
9      std::queue<simtime_t> queue2;
10     double p, q, sigma;
11     simsignal_t q1LenSignal, q2LenSignal;
12     simsignal_t q1DelaySignal, q2DelaySignal;
13     simsignal_t q1ThroughputSignal, q2ThroughputSignal;
14     cMessage *slotEvent;
15
16 protected:
17     virtual void initialize() override {
18         p = par("p");
19         q = par("q");
20         sigma = par("sigma");
21         q1LenSignal = registerSignal("q1LenSignal");
22         q2LenSignal = registerSignal("q2LenSignal");
23         q1DelaySignal = registerSignal("q1DelaySignal");
24         q2DelaySignal = registerSignal("q2DelaySignal");
25         q1ThroughputSignal = registerSignal("q1ThroughputSignal");
26         q2ThroughputSignal = registerSignal("q2ThroughputSignal");
27         slotEvent = new cMessage("slotEvent");
28         scheduleAt(0.0, slotEvent);
29     }
30
31     virtual void handleMessage(cMessage *msg) override {
32         int served1 = 0, served2 = 0;
33         bool serverAvailable = uniform(0, 1) < sigma;
34         if (serverAvailable) {
35             if (!queue1.empty()) {
36                 emit(q1DelaySignal, simTime() - queue1.front());
37                 queue1.pop();
38                 served1 = 1;
39             } else if (!queue2.empty()) {

```

```

40         emit(q2DelaySignal, simTime() - queue2.front());
41         queue2.pop();
42         served2 = 1;
43     }
44 }
45 int a1 = geometric(p);
46 for (int i = 0; i < a1; i++) queue1.push(simTime());
47 int a2 = poisson(q);
48 for (int i = 0; i < a2; i++) queue2.push(simTime());
49 emit(q1LenSignal, (int)queue1.size());
50 emit(q2LenSignal, (int)queue2.size());
51 emit(q1ThroughputSignal, served1);
52 emit(q2ThroughputSignal, served2);
53 scheduleAt(simTime() + 1.0, slotEvent);
54 }
55 };
56
57 Define_Module(ModuloCode);

```

Listing 2: ModuloCode.cc

B Topologia NED

```

1  simple ModuloCode {
2      parameters:
3          double p;
4          double q;
5          double sigma;
6          @display("i=block/queue");
7
8          @signal[q1LenSignal](type="int");
9          @signal[q2LenSignal](type="int");
10         @signal[q1DelaySignal](type="simtime_t");
11         @signal[q2DelaySignal](type="simtime_t");
12         @signal[q1ThroughputSignal](type="int");
13         @signal[q2ThroughputSignal](type="int");
14
15         @statistic[coda1Len](title="Lunghezza Coda 1";
16             source="q1LenSignal"; record=vector,mean,last;
17             interpolationmode=sample-hold);
18         @statistic[coda2Len](title="Lunghezza Coda 2";
19             source="q2LenSignal"; record=vector,mean,last;
20             interpolationmode=sample-hold);
21         @statistic[coda1Delay](title="Tempo Permanenza 1";
22             source="q1DelaySignal"; record=vector,mean;
23             interpolationmode=none);
24         @statistic[coda2Delay](title="Tempo Permanenza 2";
25             source="q2DelaySignal"; record=vector,mean;
26             interpolationmode=none);
27         @statistic[coda1Thru](title="Throughput 1";
28             source="q1ThroughputSignal"; record=mean);
29         @statistic[coda2Thru](title="Throughput 2";
30             source="q2ThroughputSignal"; record=mean);
31     }
32
33     network SistemaPriorita {

```

```

34     submodules:
35         sistema: ModuloCode;
36 }

```

Listing 3: SistemaPriorita.ned

C Configurazione omnetpp.ini

```

1  [General]
2  network = SistemaPriorita
3  sim-time-limit = 200000s
4  repeat = 20
5  warmup-period = 10000s
6
7  **.p      = ${p_val=0.75, 0.85}
8  **.q      = ${q_val=0.2, 0.3}
9  **.sigma  = ${sigma_val=0.6, 0.7}
10
11 [Config TransienteWelch]
12 network = SistemaPriorita
13 sim-time-limit = 20000s
14 repeat = 20
15 **.p = 0.75
16 **.q = 0.2
17 **.sigma = 0.6
18 **.q2LenSignal:vector.record = true

```

Listing 4: omnetpp.ini

D Script di Analisi Statistica (Python)

Lo script `analisi_risultati.py`, incluso nel repository, automatizza il parsing dei 160 file `.sca` e produce tutte le tabelle di questo documento. Per ogni configurazione (p, q, σ) raccoglie le 20 stime per metrica e applica la formula (8). L'esecuzione richiede Python 3 senza dipendenze esterne:

```

1  python3 analisi_risultati.py

```